

视觉-控制对接文档

ROS 视觉-控制对接文档 (Tracking Topic 规范)

1. Topic 基础信息

项目	内容
Topic 名称	/tracking
Message 类型	std_msgs/String
发布频率	硬件同步后与单线雷达一致
通信机制	ROS2
发布节点名称	vision_node

2. 字段详细规范 (含边界与异常处理)

字段	类型	取值范围	含义说明	控制端处理建议
x	float32	[0.0, 1.0]	归一化水平位置 (图像左→右)；若 detected ≠ 1, 该值无效 (应忽略)	检查 detected=1 后再使用, 否则视为无效数据
y	float32	[0.0, 1.0]	归一化垂直位置 (图像上→下)；同上	同上
detected	float32	-1.0 或 1.0	仅两个合法值: 1.0 → 目标被稳定检测到 -1.0 → 未检测到 (非0! 避免误判为“假阴性”)	必须严格校验: 若收到 0、2 等非法值, 视为数据异常, 丢弃本帧并告警

<code>timestamp_ms</code>	<code>float32</code>	≥ 0	毫秒级时间戳 (单位: ms) • 必须明确基准: 是 <code>ros::Time::now().toNSec()</code> / <code>1e6</code> ? 还是硬件触发时间? • 建议采用 ROS 系统时间 (wall time) , 便于跨节点同步	控制端应记录此时间戳用于: 延迟分析、时间对齐、状态机超时判断
---------------------------	----------------------	----------	--	---------------------------------

示例: 【0.521 0.1323 1 1715570000123456】
表示检测到目标, 目标位置为【0.521 0.1323】17155700.00123456为检测结果对应的时间戳

示例: 【0.0 0.0 -1 1715570000123456】
表示未检测到目标, 目标位置无效 17155700.00123456为检测结果对应的时间戳

3. 时序与可靠性要求

项目	要求	说明
最大端到端延迟	$\leq 500\text{ ms}$ (从目标出现在图像到 <code>/tracking</code> 发布)	视觉算法需满足此硬性指标, 控制端据此设计响应策略
丢包容忍度	允许偶发丢帧 ($\leq 5\%$) , 但 不可连续丢帧 > 3 帧	控制端需实现超时机制 (如: 若 <code>last_timestamp</code> 超过 200ms 未更新, 则触发丢失告警)
初始化行为	节点启动后, 控制端需要阻塞, 直到接收到视觉节点发送的帧	避免控制端因等待首帧而阻塞

4. 错误处理与异常场景 (控制端必读)

场景	视觉端行为	控制端应对策略
视觉模块崩溃/未启动	<code>/tracking</code> topic 消失	监控 topic 存活性 (<code>rostopic info /tracking</code>), 超时未出现则报错
长期未检测到目标	连续 <code>detected = -1</code> 超过 <code>5s</code>	控制端应触发“目标丢失”状态机 (如: 暂停运动、切换模式)